

Orbits, Not Trajectories:

A Theory of Memoised Coupling and Orbital Representation in Neural Sequence Models

Sirsh

March 2026 — Theory Document, v2

Abstract

The standard transformer stores per-layer attention projections (W_Q, W_K): $2Ld^2$ parameters encoding L independent coupling patterns. We replace them with a single shared metric tensor $\mathbf{M} = AB^\top$ ($2d^2$ parameters) applied to an evolving residual stream, defining the Orbital Response Network (ORN). The same $\mathbf{M}$ produces different coupling at each depth because the field state changes via L iterations of one generator (the semigroup of $\mathbf{M}$), which replaces L independent coupling matrices from the attention blocks across layers.

Preliminary results at toy scale confirm viability: the ORN matches or beats the transformer at 10M parameters with $5\times$ fewer coupling parameters. But beating the transformer is not the claim. The claim is that we have found the *generators*, not just the local trajectories. A trajectory records what coupling looked like at each depth; a generator encodes the law that produces all of them. This is a stronger form of knowledge, and it reveals structure—the approximate universality of coupling geometry across depth—that the transformer learns implicitly but never makes explicit.

Contents

1	The Core Problem: Extensional Storage	3
1.1	What the Transformer Stores	3
1.2	The Intensional Alternative	3
1.3	The Memoisation Programme	3
1.4	What the Brain Does Differently	4
2	Memoisation: The Formal Framework	4
2.1	Classical Memoisation	4
2.2	Memoising the Coupling Rule	5
2.3	Memoising the Embedding Table	5
2.4	On a GPU: Why This Matters Now	6
3	Capacity Theory: Why Memoisation Does Not Lose Complexity	6
3.1	Two Ways to Store Complexity	6
3.2	The Parameter–Compute Tradeoff	6
3.3	Theoretical Performance (TBD)	7
3.4	When Memoisation Fails	8
3.5	Bits of Coupling: A Concrete Metric	8
4	The Geometry of Orbital Representations	9
4.1	Tokens as Orbits	9
4.2	The Dynamical System View	9
4.3	Memory as Attractor, Not Storage	9

4.4	Semigroup Structure	9
5	The Stigmergic Residual Field	10
5.1	Biological Stigmergy	10
5.2	The Residual Stream as Shared Geometric Medium	10
5.3	Renormalisation Group Flow as Depth	10
5.4	The Spectral Gap–Capacity Duality	14
6	The Implicit Rotation Mechanism	15
6.1	Spectral Invariance: The Empirical Finding	15
6.2	Why the ORN Gets the Rotation for Free	16
6.3	Spectral Sharing as Architecture	16
7	The Two-System Architecture	16
7.1	Two Kinds of Knowledge	16
7.2	Perturbative Corrections Around the Orbital Path	17
7.3	Developmental Trajectory	19
7.4	Biological Analogy	19
8	Literature Review	20
8.1	Parameter Sharing and Weight Tying	20
8.2	Implicit Depth and Fixed-Point Models	20
8.3	Computed Embeddings	21
8.4	Attention Simplification	21
8.5	Riemannian Geometry of Representations	21
8.6	Information Geometry	21
8.7	Kolmogorov Complexity and Compression	22
8.8	Semigroups and Dynamical Systems	22
8.9	Geometric Deep Learning	22
8.10	Modern Hopfield Networks	22
8.11	Renormalisation Group in Deep Learning	23
8.12	Complementary Learning Systems	23
9	Experimental Evidence	23
9.1	Coupling Memoisation	24
9.2	Orbital Embeddings	24
9.3	Spectral Sharing	25
9.4	Summary	25
10	Honest Assessment	25
10.1	Future Direction: RG Diagnostics as a Measurement Framework	26
11	Conclusion	26

1 The Core Problem: Extensional Storage

1.1 What the Transformer Stores

A transformer with L layers, vocabulary V , and model dimension d stores two dominant structures:

The embedding table. A dense lookup: $E \in \mathbb{R}^{V \times d}$, mapping each token ID to a d -dimensional vector. At GPT-2 scale ($V=50,257$, $d=768$), this is 38.6M parameters—31% of the entire model. Each token’s vector is stored independently; any cluster structure (“Emma”, “Max”, “Lily” grouping near each other) is an accident of co-occurrence statistics, not a learned abstraction.

The coupling projections. At each of L layers, separate $W_Q^{(l)}, W_K^{(l)}$ matrices ($2d^2$ parameters per layer) encode the rule: “which tokens should attend to which?” At GPT-3 scale ($L=96$, $d=12,288$), this is approximately 28 billion parameters dedicated to answering one question—independently at every layer. Layer 47 does not know what layer 46 concluded. It re-derives the coupling from scratch.

Both structures are **extensional**: they enumerate instances (one vector per token, one coupling matrix per layer) rather than encoding the rules that generate them.

1.2 The Intensional Alternative

The alternative is **intensional storage**: store the compact rule that generates the instances on demand.

Definition 1 (Extensional representation). *A mapping $f : W \rightarrow \mathbb{R}^d$ that assigns each element $w \in W$ an independent vector. Space: $\mathcal{O}(|W| \times d)$. No structure is enforced; any regularity is emergent.*

Definition 2 (Intensional representation). *A mapping $f : W \rightarrow \mathbb{R}^d$ defined by a compact rule g and a minimal code c_w : $f(w) = g(c_w)$, where $\dim(c_w) \ll d$ and g is shared across all w . Space: $\mathcal{O}(|W| \times \dim(c) + |g|)$. Regularity is enforced by the shared generator g .*

In group theory terms: the integers under addition have infinitely many elements, but two generators suffice (+1 and -1). We conjecture that both the embedding structure and the coupling geometry of transformers have low generator rank—a small number of geometric primitives, composed differently by the evolving residual stream, account for the full structure.

1.3 The Memoisation Programme

The ORN applies intensional replacement to two transformer components:

Component	Extensional (former)	(Trans- Intensional (ORN))
Embedding table	Dense lookup $E[w]$, $\mathcal{O}(Vd)$	Seed $\mathbf{s}_w$ + shared dynamics $\Phi^T(\mathbf{s}_w)$, $\mathcal{O}(Vd_s + \Phi)$
Coupling	Per-layer $W_Q^{(l)}, W_K^{(l)}$, $\mathcal{O}(Ld^2)$	Shared $\mathbf{M} = AB^\top$, $\mathcal{O}(d^2)$
Response	Per-layer $W_V^{(l)}, W_O^{(l)}$	Per-layer $W_V^{(l)}, W_O^{(l)}$ (unchanged)

Response heads remain per-layer because they serve a dual role (Section 6). The savings from coupling and embedding memoisation are redistributed to model width.

1.4 What the Brain Does Differently

The brain has roughly 86 billion neurons consuming ~ 20 watts. It does not store a $50,000 \times 768$ lookup table for word representations. It does not maintain separate query-key matrices at each processing depth. Whatever it does, it is vastly more compressed than a transformer.

Memories are reconstructive, not retrieveive: every recall is a regeneration from a partial cue, not a read from a fixed address. Memories drift, merge, and confabulate. They are infected by present context. This is not a bug.

Claim 1. *The brain stores the generator of memories, not the memories themselves. Recall is a generative act—running the generator forward from a partial cue to reconstruct the full state.*

A concept like “justice” or “momentum” is not stored as a fixed vector. It is a point reachable by composing a small set of primitive generators acting on a base space. The concept is an *orbit* under the action of the generator group. You do not store the orbit. You store the generators. You recover any point by applying generators from wherever you currently are.

The transformer stores trajectory fragments—enough sampled points from enough orbits to approximate the full orbit structure by interpolation. And it works. At internet scale, with hundreds of billions of parameters and trillions of tokens, the trajectory sampling becomes dense enough that interpolation covers essentially all of natural language. The transformer does not fail; it succeeds impressively.

Our argument is not that the transformer cannot approximate the orbits. It is that approximation-by-enumeration is *expensive and bounded*. Every new orbit region requires more stored fragments. Every increase in coverage requires proportionally more parameters. The scaling laws reflect this directly: capability grows as a power law of parameters because you are buying more trajectory fragments, and each fragment covers only its local neighbourhood.

The generator, if found, changes the scaling regime. A system that stores the rule rather than the instances does not need more parameters to cover more of the orbit space—the rule already covers all of it. The cost is compute (running the rule), not storage (enumerating the outputs). The question is not whether the transformer works—it does—but whether it is the right trade. On modern hardware where compute is cheap and memory is expensive, the answer may be no.

2 Memoisation: The Formal Framework

2.1 Classical Memoisation

Definition 3 (Memoisation). *Given a pure function $f : X \rightarrow Y$, memoisation is the transformation:*

$$f_{\text{memo}}(x) = \begin{cases} \text{cache}[x] & \text{if } x \in \text{cache} \\ f(x); \text{ cache}[x] \leftarrow \text{result} & \text{otherwise} \end{cases}$$

The function is computed once per unique input; subsequent calls with the same input return the cached result.

The precondition is **referential transparency**: f must be a pure function—same input, same output. Memoisation exploits the assumption that the same computation will be requested multiple times.

2.2 Memoising the Coupling Rule

In a standard transformer, layer l computes coupling scores:

$$s_l(i, j) = \frac{\overbrace{(W_Q^{(l)} \mathbf{x}_i)^\top (W_K^{(l)} \mathbf{x}_j)}^{\text{query key}}}{\sqrt{d}} = \frac{\mathbf{x}_i^\top \overbrace{W_Q^{(l)\top} W_K^{(l)}}^{\mathbf{M}_l} \mathbf{x}_j}{\sqrt{d}} \quad (1)$$

The effective coupling matrix at layer l is $\mathbf{M}_l = W_Q^{(l)\top} W_K^{(l)}$. Memoisation asks: is $\mathbf{M}_l$ approximately the same function across layers? If so, we replace L separate matrices with a single shared $\mathbf{M}$:

$$s(i, j) = \frac{\mathbf{x}_i^\top \mathbf{M} \mathbf{x}_j}{\sqrt{d}} \quad (2)$$

We are not memoising the *scores* $s(i, j)$ —those change every layer because $\mathbf{x}_i$ and $\mathbf{x}_j$ evolve. We are memoising the *rule*: the matrix $\mathbf{M}$ that defines how any two token representations interact.

Why Q and K are two matrices. It is natural to think that W_Q and W_K serve two purposes: subspace selection and asymmetry modelling. But subspace selection never required two matrices. A single projection $W \in \mathbb{R}^{r \times d}$ suffices:

$$\text{score}_{ij} = (Wx_i)^\top (Wx_j) = x_i^\top W^\top W x_j$$

This selects an r -dimensional subspace—but $W^\top W$ is symmetric, so $\text{score}_{ij} = \text{score}_{ji}$ always. The *only* reason attention needs two projections is to break this symmetry: “what I’m looking for” (query) differs from “what I’m advertising” (key). W_Q and W_K exist because the attention graph is directed, not because subspace selection requires two matrices.

This is why $\mathbf{M} = AB^\top$ is the natural shared coupling: A and B provide asymmetry; their rank controls the subspace dimension. Symmetric $\mathbf{M} = LL^\top$ fails catastrophically on autoregressive tasks (confirmed experimentally), because it forces mutual attention in an inherently directional setting.

2.3 Memoising the Embedding Table

The same logic applies to the embedding table. $E \in \mathbb{R}^{V \times d}$ maps each token to an independent vector. But most tokens are not independent: “happy”, “glad”, “pleased” share a semantic role. The table stores V points that largely lie on a low-dimensional manifold—PCA of GPT-2 embeddings shows $\sim 90\%$ of variance in the top ~ 50 of 768 dimensions.

The intensional replacement: each token receives a minimal **seed** $\mathbf{s}_w \in \mathbb{R}^{d_s}$ ($d_s \ll d$), and a shared dynamical system Φ unfolds seeds into full representations:

$$\mathbf{e}_w = \Phi^T(\text{proj}(\mathbf{s}_w)) \quad (3)$$

where T iterations of the shared block Φ progressively expand and refine the representation.

Claim 2 (Memoised embedding). *A memoised embedding is a representation system where: (1) each token has a minimal code c_w (the seed); (2) a shared function Φ (the same dynamical system for all tokens) unfolds codes into representations; (3) the representation is $\Phi^T(c_w)$: T iterations of Φ on the code. The token’s meaning is not stored—it is computed by running a shared function on a compressed address.*

The generative view. The obvious reading of $f(D) \approx E$ is: “ D is a compressed version of E , and f decompresses it.” This misses the deeper point. D (the seed codes) is the *native* representation—the natural coordinate system of the vocabulary. f (the shared dynamics) is the physics—the shared law that unfolds compressed identity into usable representation. E (the lookup table) is the *materialised cache* of $f(D)$.

In this view, the lookup table is the approximation. The orbit is the ground truth. Training a lookup table is memoising a function you haven’t discovered yet.

2.4 On a GPU: Why This Matters Now

Modern GPU performance is dominated by **memory bandwidth**, not arithmetic throughput. An A100 has 312 TFLOPS arithmetic but only 2 TB/s HBM bandwidth: for every byte loaded, the GPU can perform ~ 156 operations.

Embedding lookup is pure memory access: load one row of E per token, zero compute. Orbital computation loads Φ once (shared across all tokens) and runs T iterations of matrix multiplications—converting a memory-bound operation into a compute-bound one.

Similarly, the standard transformer loads $2d^2$ fresh coupling parameters from HBM at every layer. The ORN loads $\mathbf{M}$ once and reuses it across all L layers. At $L=96$ layers, this is a $96\times$ reduction in coupling parameter loads.

The trade is: memory access for computation. On modern hardware, computation is cheap and memory is expensive. This is the same insight behind Flash Attention, gradient checkpointing, and mixture-of-experts: **modern hardware prefers computation over storage**.

3 Capacity Theory: Why Memoisation Does Not Lose Complexity

A natural objection: fewer parameters means less capacity. This argument is wrong, but the reason is subtle.

3.1 Two Ways to Store Complexity

Consider a function f mapping inputs to outputs. The **Kolmogorov complexity** $K(f)$ is the length of the shortest program that computes f .

Lookup table (extensional). Store every input-output pair. Space: $\mathcal{O}(|X|)$. No computation at inference.

Program (intensional). Store a compact program that *computes* $f(x)$ for any x . Space: $\mathcal{O}(K(f))$. Computation at inference—run the program.

For a random function, $K(f) \approx |X|$ —no compression possible. But for a structured function, $K(f) \ll |X|$.

Claim 3 (Capacity equivalence). *A system with P parameters implementing a program of Kolmogorov complexity K can represent any function with $K(f) \leq P$, regardless of domain size. The transformer uses $\mathcal{O}(Ld^2)$ parameters as a partial lookup table. The ORN uses $\mathcal{O}(d^2)$ parameters as a program, supplemented with T steps of computation (iteration). Capacity is preserved because parameters are traded for compute.*

3.2 The Parameter–Compute Tradeoff

This tradeoff is a theorem, not speculation.

A transformer with L layers and d^2 parameters per layer has circuit size $\mathcal{O}(Ld^2)$ and depth $\mathcal{O}(L)$. A weight-tied ORN with one block of d^2 parameters iterated T times has circuit size $\mathcal{O}(Td^2)$ and depth $\mathcal{O}(T)$.

If $T = L$, the circuit sizes are identical:

$$\text{Parameters} \times \text{Iterations} \geq C(f) \quad (4)$$

You can have many parameters and few iterations (transformer), or few parameters and many iterations (ORN). The total computational capacity is the product, not either factor alone.

Depth is valuable, but transformers cannot afford it. The scaling laws literature establishes that depth is more parameter-efficient than width for compositionality [35]: each additional layer adds compositional capacity that width cannot replicate. Kaplan et al. [33] and Hoffmann et al. [34] find that optimal model shape scales depth and width together, but within a fixed parameter budget, depth yields diminishing returns because each layer is expensive—a standard transformer layer costs $4d^2$ coupling parameters (W_Q, W_K, W_V, W_O) plus the FFN. At 10M total parameters with $d = 176$, the budget affords only $L = 3$ transformer layers.

SharedM changes this tradeoff. By sharing coupling ($2d^2$ shared vs. $4d^2$ per layer), each additional layer costs only the response heads (W_V, W_O : $2d^2$) and FFN ($2 \times 4d^2$). This makes depth *cheap*: at 10M, SharedM achieves $L = 13$ at $d = 144$, vs. $L = 3$ at $d = 176$ for the standard transformer. The freed coupling budget buys a $4\times$ increase in depth.

This may be the primary mechanism behind SharedM’s performance at 10M. Whether SharedM wins because the shared coupling is fundamentally better, or because it enables depth that the standard transformer cannot afford, is an open question (Experiment 9 tests this directly by comparing a deep-narrow transformer at $L = 13$ against SharedM at matched depth and total parameters). Either way, the architectural contribution is real: SharedM makes the depth regime accessible at smaller parameter budgets.

The analogy. ORN is to the transformer as JPEG is to bitmap. A bitmap stores every pixel: $\mathcal{O}(n^2)$ bits. JPEG stores DCT coefficients: $\mathcal{O}(K)$ bits where $K \ll n^2$ for natural images. JPEG requires computation (inverse DCT) to reconstruct. The image’s complexity is the same—the representation has changed.

3.3 Theoretical Performance (TBD)

SharedM swaps memory for compute: fewer stored parameters, more sequential iterations. This tradeoff has concrete implications for training and inference performance that we outline here theoretically; empirical benchmarking at scale remains future work.

Training throughput. At matched total parameters, SharedM and the transformer are roughly comparable in wall-clock time per step. SharedM’s blocks are cheaper (shared A, B eliminate per-layer coupling weights: $2d^2$ per layer vs. $4d^2$), but SharedM goes deeper ($L = 13$ vs. $L = 3$ at 10M). More layers means more sequential operations; cheaper layers means less work per operation. In our experiments these approximately cancel: both architectures complete 5000 steps in ~ 55 minutes at 10M on Apple M4.

Inference latency. For autoregressive generation, each token requires a full forward pass through all L layers. SharedM’s depth advantage becomes a latency cost: at 1B parameters, SharedM might achieve $L = 80$ where the transformer gets $L = 24$ —roughly $3\times$ more sequential steps. Each step is cheaper (fewer unique weights to load), but the sequential depth is the bottleneck. Time-to-first-token and tokens-per-second both degrade proportionally to the depth ratio.

Memory bandwidth. On modern accelerators (GPU, TPU), inference is typically *memory-bandwidth-bound*: the bottleneck is loading weights from HBM, not computing with them. SharedM has an advantage here: the shared A, B matrices are loaded once and reused at every layer, fitting in cache. Per-layer W_Q, W_K in the standard transformer must be loaded fresh at each layer. At high batch sizes where the compute-to-memory ratio shifts, SharedM’s smaller unique weight footprint could yield higher throughput despite more layers.

Adaptive depth as mitigation. The depth–latency cost motivates *adaptive computation*: not all tokens need the full orbital depth. If the orbit converges early (easy, predictable tokens like “the”, “and”), the model can exit early. Hard tokens (rare words, ambiguous contexts) run the full depth. This is the adaptive halting mechanism of the Universal Transformer [2] applied to SharedM, and it connects directly to our perturbative correction architecture: if the context gate $g \approx 0$ for a token at some layer, the orbital trajectory has already converged and further iteration adds nothing.

The user experience this implies: *fast for predictable text, slower for surprising text*—which mirrors how human reading works. Formalising the convergence criterion (when is the orbit “done”?) and measuring the practical speedup from early exit at scale are open problems for the experimental programme.

3.4 When Memoisation Fails

Memoisation loses capacity when the coupling rule is genuinely non-uniform across layers.

Define the **coupling variance**:

$$V = \frac{1}{L} \sum_{l=1}^L \|\mathbf{M}_l - \bar{\mathbf{M}}\|_F^2, \quad \bar{\mathbf{M}} = \frac{1}{L} \sum_{l=1}^L \mathbf{M}_l \quad (5)$$

If $V/\|\bar{\mathbf{M}}\|_F^2$ is small, memoisation incurs small loss. If large, layers need specialised coupling and shared $\mathbf{M}$ underfits. This is directly measurable on pretrained transformers—a scalar that predicts, before we build anything, how much quality memoisation will cost.

For embeddings, memoisation fails when the targets are unstructured. If V target embeddings are drawn i.i.d. from $\mathcal{N}(0, I_d)$, any orbital embedding achieving ϵ -reconstruction requires $V \times d_s + |\Phi| \geq V \times d - \mathcal{O}(V \log(1/\epsilon))$ —no compression possible. But real embeddings have massive structure: they cluster by semantic role, syntactic category, and topic, concentrating on low-dimensional manifolds.

3.5 Bits of Coupling: A Concrete Metric

Definition 4 (Coupling capacity). *The coupling capacity $\mathcal{C}$ is the number of bits required to specify the coupling rule:*

$$\mathcal{C}_{\text{transformer}} = L \times d^2 \times b \quad \text{bits} \quad (6)$$

$$\mathcal{C}_{\text{ORN}} = d^2 \times b \quad \text{bits (static)} \quad (7)$$

$$\mathcal{C}_{\text{ORN (effective)}} = d^2 \times b + T \times \underbrace{H(\mathbf{x}_t | \mathbf{x}_{t-1})}_{\text{bits per iteration}} \quad \text{bits} \quad (8)$$

where b is bits per parameter and $H(\mathbf{x}_t | \mathbf{x}_{t-1})$ is the conditional entropy of the state update.

The key insight: **iterating a simple rule on an evolving state generates bits**. A cellular automaton with 2 bits of rule (Rule 110) is Turing-complete. The ORN’s shared $\mathbf{M}$ is the rule; the evolving residual stream is the tape.

4 The Geometry of Orbital Representations

4.1 Tokens as Orbits

In the orbital framework, a token is not a stored point but a *trajectory* through a phase space defined by the shared dynamics.

Definition 5 (Phase space). *The phase space $\mathcal{M}$ is a smooth manifold equipped with a metric induced by the shared coupling tensor $\mathbf{M}$. For $\mathbf{x}, \mathbf{y} \in T_p\mathcal{M}$ (tangent vectors at point p), the inner product is $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{M}} = \mathbf{x}^\top \mathbf{M} \mathbf{y}$.*

Definition 6 (Orbital embedding). *Given a dynamical system $\Phi : \mathcal{M} \rightarrow \mathcal{M}$ and a seed $\mathbf{s}_w \in \mathcal{M}$ for token w , the orbital embedding is the trajectory: $\mathcal{O}_w = (\mathbf{s}_w, \Phi(\mathbf{s}_w), \Phi^2(\mathbf{s}_w), \dots, \Phi^T(\mathbf{s}_w))$. The representation at depth t is $\Phi^t(\mathbf{s}_w)$.*

Definition 7 (Orbit equivalence). *Two tokens w_1, w_2 are orbitally equivalent if their orbits converge: $w_1 \sim w_2 \iff \lim_{t \rightarrow \infty} \|\Phi^t(\mathbf{s}_{w_1}) - \Phi^t(\mathbf{s}_{w_2})\| = 0$. Orbitally equivalent tokens are functionally interchangeable—they reach the same attractor regardless of their surface form.*

In a children’s story dataset, “Emma”, “Max”, “Lily” should be orbitally equivalent: they fill the CHILD-CHARACTER role. Their seeds differ (the model can distinguish them), but their orbits converge (the model treats them identically for structural purposes).

4.2 The Dynamical System View

The shared block Φ defines a vector field on $\mathcal{M}$. At any point $\mathbf{x}_t = \Phi^t(\mathbf{s}_w)$, the Jacobian $J_t = \partial\Phi/\partial\mathbf{x}|_{\mathbf{x}=\mathbf{x}_t}$ describes local geometry:

- $|\lambda_i| < 1$: perturbations along eigenvector $\mathbf{v}_i$ contract. The orbit is locally stable.
- $|\lambda_i| > 1$: perturbations grow. Small seed differences amplify.
- $|\lambda_i| \approx 1$: perturbations neither grow nor shrink.

For orbital embeddings to work, we need:

- **Contracting directions** for structural features (Emma $\approx$ Max $\approx$ Lily should converge).
- **Neutral/expanding directions** for distinguishing features (Emma $\neq$ Max in surface form).

This is a *mixed-stability* regime—attractor dynamics on dissipative systems that collapse trajectories onto low-dimensional manifolds while preserving information along the manifold.

4.3 Memory as Attractor, Not Storage

Claim 4. *Retrieving a memory means running the dynamics until convergence. The attractor IS the memory. It is content-addressed by construction because the basin of attraction defines what counts as “close enough to retrieve this.”*

This connects to the modern Hopfield network view of attention [9]: attending to stored patterns is equivalent to gradient descent on an energy landscape. If the energy landscape ($\mathbf{M}$) is shared across layers, each layer performs gradient descent on the same surface from a different starting point in the evolving residual stream.

4.4 Semigroup Structure

The dynamical system Φ generates a semigroup of operators $\{\Phi^0, \Phi^1, \Phi^2, \dots\}$. Chen & Wu [16] showed that explicitly embedding the semigroup property $T(s+t) = T(s) \circ T(t)$ into neural architectures dramatically reduces data dependency and improves long-time prediction. This parallels our finding: memoisation (shared $\mathbf{M}$) works with no quality loss because we are implicitly enforcing algebraic consistency on the coupling dynamics.

The group G generated by Φ is the same for all tokens—it is determined by $\mathbf{M}$ and the shared block. Different tokens explore different parts of the same group’s action. This is the “generators, not trajectories” idea made precise: G is compact (one shared block), but the orbits $G \cdot \mathbf{s}_w$ can be arbitrarily complex.

5 The Stigmergic Residual Field

5.1 Biological Stigmergy

In biological stigmergy, agents coordinate through a shared environment. Ants deposit pheromones; subsequent ants read the field and respond. No agent needs global state. Time and space are decoupled. Specialisation emerges from simple local read-write rules.

The transformer’s residual stream maps onto this pattern:

1. Each layer reads the stream, makes a small additive contribution, and passes it on.
2. Layer k does not need to know what layer $k-3$ intended—it reads the current field state.
3. Features coexist in superposition within a single shared channel.
4. Adding layers does not require redesigning earlier layers.

The stream is the field. Attention is the rule for reading the field. We propose: **the reading rule is a property of the field geometry, not of the individual reader.**

5.2 The Residual Stream as Shared Geometric Medium

When the coupling geometry is shared ($\mathbf{M}$ is the same at every layer), the residual stream becomes more than a data bus—it becomes a **manifold with known metric structure**. Every layer, every expert, every device that reads the stream agrees on what “nearby” means, what directions are important for coupling, and how to measure relevance.

This has three practical consequences:

Mixture of experts. In a standard transformer, MoE experts write into a residual stream whose coupling geometry is defined per-layer by opaque W_Q, W_K matrices. Expert outputs compose only by accident of training. When the residual field has a shared $\mathbf{M}$, expert outputs live in a common geometric space with well-defined distances and coupling rules. Experts write perturbations into a shared manifold whose geometry is known to the coupling mechanism.

Adaptation and RAG. If the orbital core captures structural knowledge and the ontological layer (Section 7) captures domain-specific bindings, adapting to a new domain means swapping or fine-tuning the ontological layer alone. The orbital world model stays frozen. This is the natural architecture for retrieval-augmented generation: the retrieval system populates the ontological layer with context-specific bindings, while the orbital core provides structural reasoning.

Parallelisation. When processing is distributed across devices, the shared geometry provides a coordination-free protocol: each device knows the manifold structure via $\mathbf{M}$, so partial results can be merged without explicit synchronisation of coupling rules.

5.3 Renormalisation Group Flow as Depth

The renormalisation group (RG) from statistical physics provides the mathematical framework for understanding depth as coarse-graining. The RG iteratively integrates out fine degrees of freedom, obtaining an effective theory at coarser scale with fewer parameters, until convergence to a **fixed point**—a maximally compact description.

Applied to neural sequence processing:

Fine scale: tokens, subwords, local context
Medium scale: phrases, entities, local structure
Coarse scale: propositions, relations, causal structure
Abstract: symmetries, invariances, rules — the fixed point

Mehta & Schwab [13] established an exact mapping between the RG and restricted Boltzmann machines. Lin, Tegmark, & Rolnick [14] argued that deep learning succeeds because natural data has hierarchical, local, symmetric structure that RG-like operations exploit.

The standard transformer as unrenormalised field theory. In the standard transformer, each layer has its own coupling matrices $W_Q^{(\ell)}, W_K^{(\ell)}$ —its own “theory at that scale.” There is no constraint that the theory at scale ℓ should relate to the theory at scale $\ell+1$. This is analogous to computing a quantum field theory without renormalisation: evaluating the full theory at every energy scale independently. It works—you get the right answers—but it is wasteful because it does not exploit the fact that the underlying theory may be the same at every scale.

SharedM as renormalised field theory. SharedM forces the coupling rule to be the same at every scale. The *same* $\mathbf{M}$ operates at layer 1 (local correlations: “the” next to “cat”) and layer 13 (global correlations: the noun phrase’s role in the narrative).

Remark (Connections to prior work). The shared coupling geometry has structural analogues in several independent lines of work. Neural Sheaf Diffusion [37] uses shared restriction maps (transport operators on graph edges) to define a sheaf Laplacian that drives diffusion—structurally identical to SharedM in the graph setting, though sheaf Laplacians are symmetric where $\mathbf{M} = AB^\top$ is asymmetric. ALBERT [1] showed that sharing attention weights costs only -0.7 points (near zero)—empirical confirmation of coupling redundancy, though ALBERT does not analyse the geometric structure. Mingard et al. [38] showed SGD has an inbuilt Occam’s razor favouring low-complexity solutions—theoretical backing for why SGD should discover shared $\mathbf{M}$ rather than $2L$ independent matrices. MASA [39] is the closest concurrent work: it decomposes Q, K, V, O matrices into shared dictionary atoms with per-layer scalar coefficients ($\hat{W}_\ell = \sum_s c_{\ell s} \cdot D_s$), achieving 66.7% attention compression at 729M scale. Notably, MASA finds Q, K are more compressible than O (“compressing O introduces a bottleneck”), independently confirming our choice to share coupling but not response. However, MASA’s sharing is *static*—each layer independently combines atoms with no inter-layer dynamics—while the ORN’s sharing is *dynamic*: one $\mathbf{M}$ applied to an evolving field state where each layer’s coupling depends on all previous layers. MASA notes “spectral clustering is left for future work”—the spectral invariance that explains why their sharing works is exactly our contribution. What is new in the ORN is the identification of the *specific* invariant (the eigenvalue spectrum of $W_Q^\top W_K$, confirmed across five model families), the dynamic mechanism (field-state rotation by per-layer response heads), the semigroup interpretation, spectral crystallisation during training, and the transfer results showing the shared geometry generalises across domains.

What changes between layers is the field state—the residual stream has been progressively coarse-grained by previous iterations.

The analogy to a Hamiltonian is not merely rhetorical—it is structurally justified. We have constructed a semigroup-like operator ($\mathbf{M} = AB^\top$) that is shared at all scales and iterated, with per-layer response heads providing the scale-dependent transformation. This is the defining property of a Hamiltonian in the RG sense: a single rule that governs the dynamics at every scale, with the effective theory at each scale obtained by applying the rule to the coarse-grained degrees of freedom. The perturbative ontological corrections (Section 7.2) complete the picture: they are the quantum corrections around the classical path defined by the Hamiltonian, gated by

context (the boundary conditions of the path integral). The full architecture—shared generator, per-layer response, context-gated perturbations—maps onto the structure of a renormalised field theory with perturbative corrections.

In RG, the Hamiltonian is the same at every scale, but the effective degrees of freedom differ because finer-scale fluctuations have been integrated out. The coupling constants “flow” not because the rule changes, but because the system the rule acts on has changed.

Our spectral invariance finding maps onto this picture. A universality study across five pretrained models (GPT-2 at three scales, SmolLM2, Qwen2.5) confirms this is not a GPT-2 artifact: the mean spectrum correlation is **0.966** while raw cosine similarity is **0.004**. Every model tested shows > 0.93 spectral correlation with < 0.01 raw cosine—the coupling shape is invariant across layers in all trained transformers we examined.

- The **eigenvalue distribution** of $W_Q^\top W_K$ is invariant across layers (spectrum correlation 0.94–0.99 across five models). This is the *universality class*—the structural signature that is preserved under the RG flow.
- The **eigenvector orientations** differ across layers (cosine sim ~ 0.004). This is the *RG rescaling*—the coordinate rotation needed to express the same theory in the effective variables at each scale.
- The **response heads** W_V, W_O (per-layer) perform this rotation. They are the *beta functions*—describing how the effective coupling runs with scale without changing the underlying theory.

The rotation interpretation is precise: per-layer coupling matrices may not encode independent rules at all. Each layer’s response heads transform the residual stream, rotating the coordinate frame for the next layer’s coupling. Layer ℓ ’s coupling M_ℓ may be approximately $M_\ell \approx R_\ell M_1 R_\ell^\top$, where R_ℓ is the cumulative rotation from response heads 1 through $\ell-1$. Eigenvalues are preserved under similarity transformation— M_ℓ and M_1 have the same spectrum, only different eigenvectors—which is exactly the universality finding. The transformer stores L rotated copies of one coupling rule; SharedM eliminates this by sharing the rule and letting the rotation arise from the field state.

Implicit vs explicit semigroup. An important clarification: the ORN’s semigroup is *emergent*, not coded. The implementation is simple parameter sharing—one A , one B , referenced by every layer. We do not compute $\mathbf{M}^L$ or compose coupling matrices explicitly. The semigroup arises because per-layer response heads transform the field state between applications of the same $\mathbf{M}$, producing L distinct coupling patterns from one shared rule. This is architecturally identical to ALBERT’s weight sharing, except that ALBERT shares everything (including response heads) while the ORN shares only the coupling and keeps response heads per-layer. The per-layer response heads are what make the semigroup rich—they rotate the coordinate frame so that one coupling rule produces diverse patterns. Explicit semigroup computation—composing $\mathbf{M}$ in coupling space rather than through the field state—is a direction explored in the companion technical report (*Path Integrals and the ORN*), where it enables exact physics computation but does not improve language modelling.

The path integral connection. In quantum field theory:

QFT	ORN
Lagrangian (fundamental rule)	$\mathbf{M} = AB^\top$ (coupling geometry)
Path integral (sum over configurations)	Residual stream (sum of layer outputs)
Classical path (saddle point)	Orbital trajectory (SharedM forward pass)
Quantum corrections (loop diagrams)	Perturbative ontological corrections ($g \cdot \delta$)
Running coupling constants $\alpha(\mu)$	Effective attention $\text{Attn}^{(\ell)}$ varying with depth
RG fixed point	Converged orbit (late layers stabilise)
Universality class	Spectral structure of $\mathbf{M}$

Remark (The ORN is not a path integral—and should not be). Computational experiments (companion document: *Path Integrals and the ORN*) establish an important distinction. The ORN has composable coupling kernels (Chapman-Kolmogorov correlation ~ 0.99) but does not use this composability to compute the propagator T^L . Instead, deeper layers *refine* the prediction rather than *compose* it. This is the correct behaviour for contextual prediction: the path integral computes T^L , which at large L converges to the stationary distribution (averaging over all possible futures), while the ORN preserves the starting context via the residual connection and sharpens the specific prediction.

The semigroup structure of $\mathbf{M}$ serves as a **geometric scaffold**—a shared grammar for coupling at every depth—while the per-layer response heads and perturbative corrections do the actual work of context-dependent refinement. The path integral describes the grammar; the ORN speaks the language.

The complete picture (demonstrated computationally in the companion technical report *Path Integrals and the ORN*): the ORN computes a path integral along an orbit in representation space, where the “action” is determined by the shared coupling geometry $\mathbf{M}$, the “renormalisation” is performed by per-layer response heads, and the “quantum corrections” are supplied by the perturbative gate.

The coupling constants in QFT “run” with energy scale—the fine-structure constant α depends on the scale you probe. But the *theory* (QED) is the same at every scale. In SharedM: $\mathbf{M}$ is the theory. The effective coupling pattern (attention matrix) runs with depth—different at layer 2 vs. layer 10. The running is a consequence of the residual stream being progressively coarse-grained, not of the coupling rule changing.

Universality classes and domain structure. If the RG analogy is more than suggestive, it predicts a notion of **universality classes for language domains**. In physics, systems near a critical point show universal behaviour—different materials (water, magnets, percolation) exhibit the same critical exponents if they belong to the same universality class. The microscopic details do not matter; only the symmetry class does.

The analogue: different *texts* within the same domain (stories, code, legal documents) may require the same spectral structure in $\mathbf{M}$ —the same universality class of coupling geometry. “All stories share the same narrative coupling structure”—the spectral signature of $\mathbf{M}$ is the universality class. The specific tokens and their meanings are the microscopic details that get integrated out across layers.

If this analogy is real (not just suggestive), it predicts measurable observables—and we have now confirmed several of them directly on a 50M-parameter SharedM model (see companion document, *The Coupling Semigroup in Practice*):

1. **Fixed-point convergence (confirmed).** $\mathbf{M}$ should develop fixed-point structure over training: its eigenspectrum should converge to a stable configuration that reflects the universality class of the data. *Measurement:* tracking $\mathbf{M}$ ’s singular values across four checkpoints (steps 2K–8K), the effective rank sharpens from 66 to 51, the top singular value grows from 9.5 to 11.9, and asymmetry stabilises by step 4,000—well before the loss plateaus at step 6,000. The geometry crystallises first; the loss catches up.
2. **Spectral gap controls convergence rate.** The rate at which the residual stream converges across layers should relate to $\mathbf{M}$ ’s spectral gap. Let $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d$ be the singular values of $\mathbf{M}$. The spectral gap is the ratio σ_1/σ_2 (or equivalently, the difference $\sigma_1 - \sigma_2$).

When the field iterates through $\mathbf{M}$, each iteration amplifies the dominant direction (σ_1) relative to all others. After L iterations, the ratio between the projection onto the top singular direction and the second direction scales as $(\sigma_1/\sigma_2)^L$. A **large gap** ($\sigma_1 \gg \sigma_2$) means the dominant direction takes over quickly—the orbit converges in few iterations; $L = 3$ may suffice. A **small gap** ($\sigma_1 \approx \sigma_2$) means many directions compete and the field

takes many iterations to settle; deep networks ($L = 13+$) are needed.

This is precisely the **correlation length** ξ from statistical physics. In a spin system, the correlation length is related to the spectral gap of the transfer matrix by $\xi = -1/\ln(\sigma_2/\sigma_1)$. Large gap $\rightarrow$ short correlation length $\rightarrow$ the system reaches equilibrium quickly. Small gap $\rightarrow$ long correlation length $\rightarrow$ many coarse-graining steps are needed. The number of SharedM layers L plays the role of the number of RG steps, and $\mathbf{M}$'s spectral gap determines how many are needed.

3. **Phase transitions in training.** As the model discovers the right universality class, there should be a sharp transition in $\mathbf{M}$'s spectrum—analogous to the order parameter emerging at a critical point. Before the transition, $\mathbf{M}$'s spectrum is diffuse (no clear structure). After the transition, a few dominant directions emerge (the universality class crystallises). This would be visible as a sudden increase in $\mathbf{M}$'s condition number during training.
4. **Domain transfer within and between universality classes (confirmed).** Transfer should be easier within a universality class than between classes, because $\mathbf{M}$ captures the class, not the instances.

Experimental confirmation: a 50M ORN trained on OpenWebText, with $\mathbf{M}$ frozen, transfers to WikiText-103 at **1.3 nats better zero-shot** than a transformer with frozen W_Q , W_K (5.96 vs 7.24). After fine-tuning, the ORN reaches 4.19 vs the transformer's 4.78—a 0.59 nat advantage. Transfer to Python code is even stronger: ORN zero-shot 3.89 vs transformer 5.93 (**2.04 nats better**). After fine-tuning, ORN reaches 1.94 vs transformer's 2.39—a 0.45 nat advantage that persists. Code's structural regularity (brackets, scope, function calls) aligns closely with $\mathbf{M}$'s learned coupling geometry, producing the largest zero-shot transfer advantage of any domain tested.

$\mathbf{M}$'s effective rank on OWT is 71 (vs 51 on TinyStories)—the coupling geometry is richer for more complex data, consistent with the “universality class” interpretation: OWT spans more coupling dimensions than a single narrative domain. The claim is that *structurally similar* texts should share a spectral signature in $\mathbf{M}$. The effective rank of $\mathbf{M}$ is the order parameter that distinguishes classes. How many universality classes language has, and how they relate to genre, register, and domain, is an empirical question—but the transfer results confirm that web text and code belong to *overlapping* classes (shared $\mathbf{M}$ transfers between them).

Prediction 1 is now confirmed at 50M scale; predictions 2–4 remain testable with current infrastructure. Additionally, the semigroup analysis revealed two unpredicted phenomena: (a) Q-vectors trace smooth orbits through coupling space (adjacent-layer cosine ~ 0.9 , decaying to ~ 0.3 across the full depth), confirming that $\mathbf{M}$ acts as a continuous generator rather than a discrete lookup; and (b) coreference (“Lily” = “She”) emerges in K-space without supervision—co-referent tokens develop similar key representations at deeper layers, providing direct evidence for the world-model interpretation of $\mathbf{M}$. Prediction 2 requires measuring per-layer residual stream convergence alongside $\mathbf{M}$'s spectral gap. Prediction 4 requires training on multiple domains and comparing $\mathbf{M}$'s spectrum across them.

5.4 The Spectral Gap–Capacity Duality

The spectral gap controls not only convergence rate but also *capacity*—and these two effects are in tension.

The duality. Consider $\mathbf{M}$'s singular value spectrum $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d$:

Huge gap ($\sigma_1 \gg \sigma_2 \gg \dots$): Fast convergence—the orbit collapses onto a 1-dimensional subspace in few iterations. But low capacity: all tokens are driven toward the same dominant direction. The semigroup's image after L iterations is essentially 1-dimensional. The model converges fast but cannot distinguish between tokens.

No gap ($\sigma_1 \approx \sigma_2 \approx \dots \approx \sigma_d$): Slow convergence—many directions compete and the orbit takes many iterations to settle. But high capacity: $\mathbf{M}$ supports a d -dimensional orbit space. Every direction is equally weighted; the semigroup’s image is the full space. The model can distinguish everything but needs enormous depth to converge.

Graded spectrum ($\sigma_1 > \sigma_2 > \dots$, **with a tail**): The sweet spot. A few dominant directions provide fast convergence for the primary structural axes (noun/verb/determiner). A tail of smaller but non-negligible singular values provides capacity for finer distinctions (proper nouns, rare constructions, domain-specific patterns). The semigroup’s image after L iterations is a structured manifold: low-dimensional along the fast directions, with residual extent along the slow directions.

This is what we observe in practice. GPT-2’s coupling matrices have a few large eigenvalues followed by a long tail—the graded spectrum. Our trained SharedM models develop similar spectral profiles (condition number > 10 , non-trivial rank).

The fundamental tradeoff: depth buys capacity. A model with a moderate spectral gap needs depth to resolve the slow directions—the ones with smaller singular values that carry the fine-grained distinctions. Each additional layer gives the semigroup one more iteration to amplify these weaker directions. After L iterations, the effective capacity is determined by how many singular values satisfy $\sigma_i^L > \epsilon$ for some task-relevant threshold ϵ .

This reframes the SharedM depth advantage. SharedM’s deeper architectures ($L = 13$ vs. $L = 3$) are not merely more compositional—they exploit more of $\mathbf{M}$ ’s spectral capacity. The slow singular directions, which carry the fine distinctions, only become active after enough iterations. The standard transformer at $L = 3$ has access to only the top few directions of its (per-layer) coupling matrices; the long tail is wasted. SharedM at $L = 13$ has access to the full graded spectrum.

The condition number as a diagnostic. $\mathbf{M}$ ’s condition number $\kappa = \sigma_1/\sigma_d$ measures the span of the graded spectrum.

- $\kappa \approx 1$: $\mathbf{M}$ is near-isotropic. No preferred directions, no structure. Kill criterion B3 (condition number < 2) flags this— $\mathbf{M}$ has collapsed to a scaled identity.
- $\kappa = 10$ – 100 : healthy graded spectrum. A few dominant structural axes plus a tail of finer distinctions. The semigroup has moderate capacity, exploitable with moderate depth.
- $\kappa > 1000$: degenerate. The dominant direction overwhelms everything; the tail is numerically negligible. The semigroup’s image is effectively 1-dimensional regardless of L .

The condition number of a trained $\mathbf{M}$, logged over training, should exhibit the phase transition predicted above: diffuse early (low κ), then a sharp increase as the universality class crystallises, settling at a value that reflects the structural complexity of the domain.

If the universality class picture is correct, it implies that the orbital programme is not merely an architectural optimisation but a discovery about the *structure of language processing itself*: coupling geometry has universality classes, the spectral gap determines the depth-capacity tradeoff, and the transformer’s per-layer parameterisation obscures this by storing each layer’s representation of the same universal structure independently.

6 The Implicit Rotation Mechanism

6.1 Spectral Invariance: The Empirical Finding

Analysis of trained transformers reveals a refined picture of coupling invariance. Raw coupling matrices $\mathbf{M}_l = W_Q^{(l)\top} W_K^{(l)}$ differ across layers: cosine similarity of flattened matrices averages 0.19—well below naïve sharing thresholds. But their *eigenvalue spectra* correlate at > 0.97 across all layers and heads.

The coupling *shape* (eigenvalues) is invariant; the coupling *orientation* (eigenvectors) rotates with depth. The coupling rule is: “tokens of type X attend to tokens of type Y with strength λ_k .” This rule is the same at every layer. But the direction encoding “type X” in activation space drifts as the residual stream evolves.

6.2 Why the ORN Gets the Rotation for Free

The per-layer eigenvector rotation is not an independent parameter. It is a *consequence* of the residual stream evolving under the per-layer response heads ($W_V^{(l)}, W_O^{(l)}$) and FFNs.

When layer l ’s response heads update the residual stream, they rotate the coordinate frame that layer $l+1$ ’s shared $\mathbf{M}$ will operate in. The coupling rule is fixed; the field it acts upon evolves; and the rotation emerges from that evolution.

The dual role of response heads. In the ORN, per-layer response heads serve two functions simultaneously:

1. **Content transformation** (explicit): given a neighbourhood determined by $\mathbf{M}$, extract the relevant information at this depth.
2. **Coordinate rotation** (implicit): by updating the residual stream, steer the field into the orientation that the shared coupling needs at the next layer.

The coupling and response *co-evolve* through the shared residual field: the coupling shapes the attention pattern, the response updates the field, and the updated field changes how the same coupling is expressed at the next layer.

This explains why response heads benefit from per-layer specialisation ($\sim 4\%$ loss when tied) even when coupling is shared: they carry the additional burden of implicit coordinate alignment. It is also a design constraint: response heads in the ORN do *more work* than their transformer counterparts.

6.3 Spectral Sharing as Architecture

The spectral finding motivates a factored coupling architecture:

- **Shared:** d eigenvalues Λ (the coupling strengths)
- **Per-layer:** rotation matrix R_l (aligning coupling to current residual orientation)

Cost: $\mathcal{O}(d)$ shared eigenvalues + $\mathcal{O}(d^2)$ per-layer rotation, vs. $\mathcal{O}(d^2)$ per-layer for full coupling.

CPU gap tests confirm this is essentially lossless: spectral sharing reduces coupling reconstruction error by $> 99.9\%$ compared to full matrix sharing.

However, the implicit rotation mechanism means an explicit R_l may be unnecessary in the full ORN. Whether the emergent rotation is sufficient, or whether cheap explicit corrections provide additional benefit, is an empirical question. The current evidence (shared $\mathbf{M}$ beating the transformer on Shakespeare and TinyStories) suggests the implicit mechanism works.

7 The Two-System Architecture

7.1 Two Kinds of Knowledge

Natural language mixes two fundamentally different kinds of information:

Structural. “A character wanted something, faced an obstacle, resolved it.” This is orbital—it is a path through narrative phase space, invariant across specific instances. No amount of geometric iteration derives that the character is Caesar or the date is 49 BC.

Referential. “Caesar crossed the Rubicon in 49 BC.” This is discrete—it binds a specific name to a specific event. A dense embedding table handles both, badly: it wastes capacity on redundant structural representations in order to store the occasional specific fact.

7.2 Perturbative Corrections Around the Orbital Path

The analogy to path integrals is precise. The semigroup generated by $\mathbf{M}$ defines the **classical path**—the dominant trajectory through coupling geometry. Without context, this is the best the model can do: the generic structural prediction (“a noun follows this preposition”). Context provides the boundary conditions that select a specific trajectory from the family of reachable orbits.

The orbital model as mean field. The SharedM forward pass produces what we might call the **mean-field prediction**: the most probable trajectory given only the structural rules encoded in $\mathbf{M}$. “The cat sat on the _____” $\rightarrow$ the orbit produces the generic noun-after-preposition region, where “mat”, “floor”, “chair” are roughly equiprobable. This is correct—and sufficient if we have no further context.

The perturbation: context-dependent corrections. When preceding text provides specific context (“the cat sat on the warm red _____”), the prediction should *tighten*: “mat” should be boosted, “floor” suppressed. The orbit was already in the right neighbourhood; the perturbation provides a small, context-dependent correction that sharpens the prediction.

The architecture this implies:

$$\mathbf{x}^{(\ell)} = \mathbf{x}_{\text{orbital}}^{(\ell)} + g(\mathbf{x}^{(\ell)}) \cdot \delta(\mathbf{x}^{(\ell)}) \quad (9)$$

where $\mathbf{x}_{\text{orbital}}^{(\ell)}$ is the standard SharedM layer output, $\delta(\cdot)$ is a small correction network (MLP), and $g(\cdot)$ is a **context gate** that controls the perturbation magnitude.

Why additive corrections are natural. This is not an exotic operation—it is how transformers already work. Every layer in a standard transformer is a residual connection: $\mathbf{x} = \mathbf{x} + \text{attn}(\mathbf{x}) + \text{ffn}(\mathbf{x})$. The residual stream is a space where additive contributions accumulate from many sources. Our perturbation is one more additive term in this stream. The key difference is that we make it *explicit, gated, and separately controllable*, rather than entangling it with the attention and FFN outputs.

In principle, the correction can be generalised to a *sum* of independent perturbations, each addressing a different aspect:

$$\mathbf{x}^{(\ell)} = \mathbf{x}_{\text{orbital}}^{(\ell)} + g_{\text{ctx}}(\mathbf{x}) \cdot \delta_{\text{ctx}}(\mathbf{x}) + g_{\text{entity}}(\mathbf{x}) \cdot \delta_{\text{entity}}(\mathbf{x}) + \dots \quad (10)$$

Each gate independently controls whether its correction fires. This opens a natural surface for fine-tuning and adaptation: freeze the orbital core ($\mathbf{M}$ and response heads), add a new correction term for a new domain or task. Different corrections can be composed, swapped, or ablated independently—a modular architecture where structure (orbital) and specificity (perturbations) are cleanly separated.

The context gate is the key design element. The gate $g : \mathbb{R}^d \rightarrow \mathbb{R}^+$ reads the residual stream—which encodes accumulated context from prior tokens via $\mathbf{M}$ ’s coupling—and outputs a scalar controlling how much correction to apply:

- **No context** (first token, generic position): $g \approx 0$. Pure orbital output. The model samples from the structural prior.

- **Rich context** (late in sequence, specific preceding tokens): $g > 0$. The perturbation tightens the prediction beyond what the orbit alone provides.

This behaviour is *desired*: without context, we do not care about specific details—a sample from the orbit is the honest answer. As context accumulates, the prediction should tighten. The perturbation is what provides this tightening.

Empirical confirmation at 10M scale. Exp 8 tested the perturbative design at 10M parameters on TinyStories (5,000 steps, external machine). The perturbative model was forced to sacrifice a full layer (L=4 vs SharedM’s L=5) to match total params. Despite losing 20% of its orbital depth, it **recovered to match SharedM** by step 2,500 and was slightly ahead by step 4,000 (−0.010 nats).

All three developmental predictions from Section 7.2 are confirmed:

1. **Gate opens over training:** $\bar{g}$ increased from 0.05 (step 1) to 0.36 (step 4,000), then plateaued. The corrections start dormant and activate as the orbital core matures.
2. **Perturbation stays small:** $\|\delta\|/\|\mathbf{x}_{\text{orbital}}\|$ settled at ~ 0.57 after an initial overshoot (0.80 at step 500). The effective contribution is $g \times \delta/h \approx 0.20$ —a genuine correction, not a replacement.
3. **Gate tracks attention confidence:** gate magnitude anticorrelates with attention entropy. The gate opens when attention sharpens (low entropy = confident, context-rich coupling) and stays closed when attention is diffuse. This was first observed at CPU toy scale ($r = -0.95$) and is confirmed at 10M.

The crossover dynamics match the predicted developmental trajectory: early training develops the orbital core (corrections dormant); mid training shows the perturbative model falling behind (missing layer hurts); late training shows corrections catching up as the gate opens and context-dependent refinement activates. This is the same “structural investment pays off late” signature seen in SharedM vs transformer (Exp 3, crossover at step $\sim 2,800$).

An ablation study on the trained perturbative model provides the strongest evidence: disabling the corrections at inference (forcing $g=0$) increases val loss from 2.755 to 3.204—a gap of **0.449 nats**. This is larger than the entire improvement from step 1,000 to step 5,000 for SharedM alone. The corrections are not a minor refinement; they carry nearly half a nat of the model’s predictive capacity.

Per-position analysis confirms the context-dependence prediction (iii): late positions benefit more (+0.440 nats) than early positions (+0.392 nats), though the early benefit is also substantial. Critically, the ablation reveals that the orbital backbone *co-adapted* with the corrections—it offloaded work to the correction pathway and reorganised accordingly. The perturbative model learned a different orbital solution than standalone SharedM.

The matched-depth follow-up (Exp 8b: same backbone d=168, L=5 as SharedM, corrections as +1.1% extra params) is the clean test that will determine whether corrections genuinely add value beyond compensating for lost depth.

Constraints.

1. The perturbation acts on the **value stream only**—it modifies what information flows through the coupling, never the coupling geometry itself. **M** remains the sole authority on who attends to whom.
2. The perturbation is **small**—it is a correction around the classical path, not an alternative path. If $\|\delta\| \gg \|\mathbf{x}_{\text{orbital}}\|$, the perturbation has overwhelmed the orbital signal and the architecture has degenerated into a standard network with extra steps.
3. The perturbation is **context-dependent, not token-dependent**—it reads from the residual stream (which carries contextual information), not from a per-token lookup table. The same token in different contexts should receive different corrections.

Relation to prior onto design. Our earlier experiments (Exp 4d, Exp 7) used a parallel ontological attention pathway with its own W_Q, W_K, W_V, W_O at rank $r=4$. This was architecturally wrong: it introduced competing coupling alongside $\mathbf{M}$ and consumed parameter budget without contributing proportionally (onto_gap $\sim 1.5\%$ at 10M). The perturbative design eliminates the separate coupling entirely—the correction is a residual perturbation on the SharedM output, gated by context, with no attention mechanism of its own.

7.3 Developmental Trajectory

Under the perturbative design, training should exhibit:

1. **Early training:** the orbital core discovers coupling geometry. The gate g is near zero because the residual stream carries little useful context. The perturbation contributes nothing—training focuses on $\mathbf{M}$.
2. **Mid training:** the orbital core is functional. Context begins to accumulate meaningfully in the residual stream. The gate opens; the perturbation begins learning context-dependent corrections.
3. **Late training:** the orbital core handles structure; the perturbation handles specificity. The gate magnitude is a learned function of context richness.

Measurable signatures: (i) the gate magnitude $\bar{g}$ should increase over training as context representations mature; (ii) the perturbation norm $\|\delta\|$ should remain small relative to $\|\mathbf{x}_{\text{orbital}}\|$; (iii) ablating the perturbation (setting $g=0$) should hurt late-sequence tokens more than early-sequence tokens.

7.4 Biological Analogy

This maps to the cortex–hippocampus division [15]:

	Cortex (orbital)	Hippocampus (ontological)
Stores	Generators, attractors, rules	Specific episodes, bindings
Updates	Slowly (gradient descent)	Rapidly (one-shot binding)
Representation	Continuous, distributed	Sparse, localist
Failure mode	Can’t remember names	Can’t generalise

The transformer collapses both into a single system (dense weight matrices) and asks gradient descent to sort out the mix. This works—transformers are powerful—but it is inefficient: the storage format is wrong for both jobs.

A hemispheric parallel. The cortex–hippocampus division is a memory-system distinction. McGilchrist [31, 32] identifies a deeper complementarity at the level of *modes of knowing*: the brain’s right hemisphere engages the world holistically—grasping context, relationship, pattern, the interconnectedness of things—while the left hemisphere excels at narrow focus, categorical precision, and the manipulation of things already known.

The mapping to our architecture is structural:

	Right hemisphere / Orbital	Left hemisphere / Ontological
Mode	Holistic, relational, contextual	Analytic, categorical, literal
Learns	Structural rules, generators	Specific bindings (names, dates, facts)
Sees	“What kinds of things exist”	“Which specific things are present”
Failure	Cannot pin down “Caesar, 49 BC”	Cannot generalise or see the whole
Role	Primary world model	Serving complement

The parallel extends to the developmental trajectory: the ontological layer starts dominant (bootstrapping the orbital core, as the left hemisphere scaffolds early learning) and thins as the

orbital core matures—the right hemisphere providing the ground from which the left operates as a focused instrument.

McGilchrist argues that the highest forms of thought—metaphor, irony, moral reasoning—require integration of both modes. Metaphor requires grasping structural similarity (orbital/right) across domains whose specific contents differ (ontological/left). Understanding that “Juliet is the sun” requires both the relational insight (warmth, centrality, life-giving) and the knowledge that Juliet and the sun are distinct entities.

This is speculative—McGilchrist’s distinction concerns modes of attention and engagement with reality, not information formats, and the biological hemispheres are far messier than our clean architectural split. But the convergence is suggestive: any system that must both generalise over structure and bind specific instances seems to require at least two complementary representations. Whether this reflects deep necessity or recurrent engineering convenience remains open.

8 Literature Review

8.1 Parameter Sharing and Weight Tying

ALBERT [1] demonstrated that sharing all parameters across BERT layers causes surprisingly small quality loss (2–3%) while reducing parameters 4–12×. Direct evidence that per-layer parameterisation is largely redundant. ALBERT also introduced factorised embedding parameterisation ($V \times d_s + d_s \times d$ instead of $V \times d$), a direct predecessor to our orbital embedding approach—though ALBERT uses a linear projection where we use nonlinear iterative dynamics.

Universal Transformer [2] tied weights and added Adaptive Computation Time [3]. The crucial insight: weight tying forces the model to learn a reusable rule rather than specialised per-layer transformations. Results were mixed: improvements on algorithmic tasks but regression on standard language modelling.

What remains open: Both tie *all* weights. We propose tying only the coupling geometry while allowing response heads to vary—a more surgical sharing that targets the specific redundancy.

8.2 Implicit Depth and Fixed-Point Models

Deep Equilibrium Models (DEQ) [4] find the fixed point directly via root-finding, equivalent to infinite depth with finite parameters. DEQs matched transformer quality on WikiText-103 with 88% memory reduction, demonstrating that weight-tied iteration works at meaningful scale. They suffer training instability and expensive implicit differentiation.

Neural ODEs [5] parameterise layer transformations as continuous-time ODEs with $\mathcal{O}(1)$ memory via the adjoint method. Recent Neural ODE Transformers [30] parameterise transformer weights as continuous functions of depth, treating the layer index as a time variable.

Relevance: All four support our proposal: weight sharing across depth works when the underlying dynamics are approximately invariant. Our orbit concept is a variant; the novel element is that the *coupling geometry* is shared and defines the attractor landscape, not just the transformation.

8.3 Computed Embeddings

Fourier-based token embeddings [18] replace the learned embedding table with a deterministic Fourier basis plus a small MLP. Token representations are *computed* as a smooth function of token ID rather than stored. This validates the core principle that structured computation can replace explicit storage for embeddings, though the mechanism differs from ours (fixed Fourier basis vs. learned iterative dynamics).

Engram [19] argues that some information is better *stored* than computed: factual knowledge (“Caesar crossed the Rubicon in 49 BC”) has no generative structure to exploit. Their Sparsity Allocation Law shows that replacing $\sim 20\%$ of parameters with hash-based lookups *improves* performance. This is evidence *against* full orbital replacement and *for* our two-system architecture: use orbits for structure, retain sparse lookup for genuinely discrete facts.

8.4 Attention Simplification

Synthesizer [6] found that *random* attention patterns performed surprisingly well, suggesting learned coupling carries less information than assumed.

AFT [7] and **Linear Attention** [8] simplify the attention mechanism but retain per-layer parameterisation.

What these miss: All simplify the mechanism but do not address whether the coupling *rule* is invariant across layers. Our approach is orthogonal: keep the mechanism, share the geometry.

8.5 Riemannian Geometry of Representations

Hauser & Ray [20] established that residual networks perform geometric transformations on data manifolds, with learned weights defining the local metric. Our shared $\mathbf{M}$ is exactly such a metric: it defines the coupling geometry that all tokens share.

Training shapes geometry. Recent work [21] demonstrates that training *breaks the symmetric metric* of initialisation, developing anisotropic structure precisely where it matters for the task. This matches our observation: $\mathbf{M}$ develops non-trivial spectral structure (condition number > 10) via standard SGD—a geometric signature of learned structure, not an artefact.

Singular Riemannian geometry. Benfenati & Marta [22] extended this to the degenerate metrics that arise at saddle points and representation collapse regions—precisely the regimes our orbital dynamics must navigate. Katsman et al. [23] explicitly enforce manifold structure in residual updates, supporting the principle that geometric priors help.

Intrinsic dimensionality. Ansuini et al. [24] showed that intrinsic dimensionality of representations *progressively decreases* through network layers—the network compresses data onto lower-dimensional manifolds. This is direct evidence that trained embeddings have low intrinsic dimension and should be compressible via our orbital factorisation.

8.6 Information Geometry

Amari [25] showed that the parameter space of neural networks is a Riemannian manifold with the Fisher information metric, and that natural gradient (steepest in this geometry) avoids plateaus that trap ordinary SGD. If the orbital model maps seeds to probability distributions over next tokens, $\mathbf{M}$ can be interpreted as an approximation to the Fisher metric on this statistical manifold—connecting coupling geometry to the information-theoretic structure of the language.

8.7 Kolmogorov Complexity and Compression

Shaw et al. [26] formally bridge Kolmogorov complexity and transformers, constructing asymptotically optimal description length objectives by mapping transformer weights to prefix Turing machines. Their result proves transformers can achieve compression quality comparable to $K(x)$ up to an additive constant—validating our search for compressed representations.

Lotfi et al. [27] show that models which compress training data more achieve better generalisation, obtaining non-vacuous generalisation bounds for actual LLMs. This directly supports our hypothesis: the ORN’s structural compression (shared generators vs. stored trajectories) should predict better generalisation at matched parameter count.

Grokking and complexity dynamics. De Moss et al. [28] reveal that networks first memorise (high description length) then compress (low description length)—training actively reduces complexity by finding structure. Our spectral invariance finding (cosine sim 0.19 but spectrum correlation 0.98) is exactly this signature: the *structure* is shared even when the *representation* differs.

8.8 Semigroups and Dynamical Systems

Chen & Wu [16] showed that explicitly embedding the semigroup property $T(s + t) = T(s) \circ T(t)$ into neural architectures dramatically reduces data dependency and improves long-time prediction. This parallels our finding: memoisation (shared $\mathbf{M}$) works with no quality loss because we are implicitly enforcing algebraic consistency on the coupling dynamics.

Vacher [29] formalises deep networks as iterated function systems (IFS), proving existence of invariant measures under contractivity and deriving Wasserstein generalisation bounds. The orbital embedding is an IFS: the shared block Φ is the iterated map, the seeds $\mathbf{s}_w$ are initial conditions, and the attractors correspond to semantic roles.

8.9 Geometric Deep Learning

Group equivariant networks [10, 11] enforce symmetries structurally. This is the “generators, not trajectories” idea applied to spatial symmetries.

Lie group methods [12] parameterise symmetry groups directly and learn which symmetries the data respects. The learned Lie algebra generators are a compact description of the data’s invariance structure.

Relevance: Our shared $\mathbf{M}$ is a learned inner product on the representation space—a Riemannian metric whose eigenvectors define the symmetry axes of the coupling geometry. The connection to geometric deep learning is direct: we are learning the geometry of the representation space, not the attention patterns that happen to arise from it.

8.10 Modern Hopfield Networks

Ramsauer et al. [9] showed transformer attention is equivalent to a modern Hopfield update: attending to a key is converging toward a stored pattern. If the energy landscape ($\mathbf{M}$) is shared across layers, each layer performs gradient descent on the same surface from a different starting point. This is exactly the “memoised coupling rule applied to an evolving state” picture.

8.11 Renormalisation Group in Deep Learning

Mehta & Schwab [13] established an exact mapping between the RG and restricted Boltzmann machines—deep networks implement coarse-graining. This is an analytical equivalence for a specific architecture (RBMs) on statistical physics data. It does not prescribe architecture for transformers or identify what should be shared.

Lin, Tegmark, & Rolnick [14] argued that deep learning succeeds because natural data has hierarchical, local, symmetric structure that RG-like operations exploit. This provides theoretical justification for why depth works but does not identify specific architectural consequences for attention.

Li & Wang (2018) trained neural networks to *perform* RG transformations on lattice models—the network is a tool for physics, not the reverse. Lenggenhager et al. (2020) extended this to optimal RG schemes via neural networks.

Roberts et al. (2022) developed a field-theoretic framework for deep networks at infinite width, connecting neural network statistics to Gaussian processes. This analyses networks at initialisation and large width, not the depth-sharing structure we study.

How we differ. All prior work uses RG as a *lens to understand* deep learning—an analytical tool applied post hoc. We use RG as a *design principle*: the well-known redundancy of attention coupling across layers (Lan et al. 2020, Bhojanapalli et al. 2021, Mu et al. 2024), which we confirm at the spectral level, reveals that the coupling rule is an approximate *generator*. We exploit this architecturally by sharing the rule ($\mathbf{M}$) while letting the field state provide scale-dependent variation. The response heads are identified as the beta functions that run the effective coupling with depth. The spectral structure of $\mathbf{M}$ is identified as a universality class signature. This is RG-informed architecture, not RG-interpreted analysis.

The closest prior work architecturally is the Universal Transformer [2], which shares all weights across layers. But UT is motivated by Turing completeness, not by RG or spectral invariance. It does not decompose into coupling (shared) and response (per-layer), does not measure spectral invariance, and does not connect to universality classes. Our contribution is the identification of *what* should be shared (coupling geometry) and *why* (it is the scale-invariant quantity—the universality class of the RG flow).

8.12 Complementary Learning Systems

McClelland, McNaughton, & O'Reilly [15] argued that the hippocampus and neocortex are complementary learning systems: the hippocampus acquires specific episodes rapidly via one-shot binding, while the neocortex slowly extracts statistical structure through gradient-like consolidation. Neither alone is sufficient.

This maps directly to our orbital (cortex-like, structural, slow-learning) and ontological (hippocampus-like, specific, fast-binding) decomposition. The key insight is that the two systems have different *inductive biases for different kinds of knowledge*, not merely different learning rates.

9 Experimental Evidence

The experimental programme tests each claim independently. Full details are in the companion experimental paper; here we summarise results concisely.

9.1 Coupling Memoisation

Exp 0 — Coupling invariance analysis. Extracted $\mathbf{M}_{l,h} = W_Q^{(l,h)\top} W_K^{(l,h)}$ from a trained 8-layer transformer. Raw matrices differ (cosine sim 0.19), but eigenvalue spectra correlate at > 0.97 . The coupling *shape* is invariant; only the *orientation* varies. Spectral sharing (shared eigenvalues + per-layer rotations) reduces reconstruction error by $> 99.9\%$.

Exp 1 — Colour matching. Synthetic task with provably invariant coupling. Shared $\mathbf{M}$ beats per-layer W_Q, W_K by 0.7% with $16\times$ fewer coupling parameters. Tying response heads costs $\sim 4\%$ —they benefit from per-layer specialisation.

Exp B1 — Shakespeare. Character-level language modelling. ORN-Asym ($\mathbf{M} = AB^\top$) beats the transformer by 2.6% with $6\times$ fewer coupling parameters. Symmetric $\mathbf{M} = LL^\top$ is 3.6% worse—asymmetry is essential for autoregressive tasks. $\mathbf{M}$ develops rich geometry (condition number $\sim 10^5$) through standard SGD.

Exp 2 — Scaling protocol. Five parameter budgets (27K–550K). $\beta_{\text{MEMOISE}} = 0.164$ vs. $\beta_{\text{STORE}} = 0.160$ ($1.02\times$). MEMOISE matches or beats STORE at 4/5 scales; advantage trends upward with scale.

9.2 Orbital Embeddings

Exp 5a — TinyStories at 2M scale. First end-to-end test on real language ($V=50,257$, GPT-2 tokeniser). Orbital-T1 ($d_{\text{seed}}=16$, $T=1$) beats the baseline by 7.7% on validation loss. Freed embedding parameters enable $6.7\times$ wider architecture ($d_{\text{model}} = 268$ vs. 36). No curriculum, no scaffold, no auxiliary losses—seeds and dynamics co-learn from random initialisation.

Exp 5b — TinyStories at 10M scale. The fair test: the baseline now has $d_{\text{head}} = 44$ (genuinely functional). Orbital-T1 ($d = 476$, $L = 3$) reaches val loss 3.008 vs. baseline 2.921 (-3.0%). LOWRANK ($T=0$, no dynamics) degrades to -33.8% , confirming that compression alone fails at scale—the dynamics Φ recover 91% of the bottleneck. Width beats depth: T1 ($d = 476$, $L = 3$) outperforms T2 ($d = 296$, $L = 8$), consistent with the unfolding hypothesis (the initial projection from $d_{\text{seed}} = 16$ needs room to expand). The residual 3% gap traces to seed dimensionality ($d_{\text{seed}} = 16$ captures only 50% of transition variance), not dynamics quality.

Honest assessment. Orbital embeddings are currently the weakest component of the ORN story. The 2M win ($+7.7\%$) is dominated by parameter reallocation (baseline is pathologically narrow at $d=36$); the 10M result (-3.0%) is a fairer test, and the orbital model *loses*. Subsequent ablation (Exp 5c) shows $d_{\text{seed}}=64$ beats the baseline by 4.7%, but the winning ingredient appears to be the expansion ratio ($d/d_{\text{seed}} \approx 5$), not necessarily the iterative dynamics. The critical missing experiment is **LowRank at $d_{\text{seed}}=64$ with $T=0$** : if a linear projection from 64-dimensional seeds matches the orbital result, the dynamics contribute nothing and orbital embeddings reduce to a bottleneck factorisation. Furthermore, at 1B+ scale, embeddings are $<1\%$ of the model and the compression problem disappears naturally—SharedM savings (proportional to L) grow with depth, while embedding savings become negligible. We position orbital embeddings as an *optional extension* of the core ORN, not a co-equal pillar. The core innovation is the shared coupling geometry $\mathbf{M} = AB^\top$.

CPU gap tests. Three targeted theory validations: (1) Small MLPs perfectly reconstruct structured targets (line, circle, spiral) but fail on random targets—confirming the structure/compression distinction. (2) Nonlinear factorisation (MLP) outperforms linear (SVD) in 23/30 cases on

clustered targets. (3) $d_{\text{seed}} = 16$ captures only 50% of bigram transition variance, suggesting $d_{\text{seed}} = 32\text{--}64$ would improve results.

9.3 Spectral Sharing

CPU gap test 6 — Per-layer rotation. Generated coupling matrices matching Exp 0’s properties (spectrum correlation ~ 0.98 , cosine sim ~ 0.19) and tested three sharing strategies: full sharing (baseline), low-rank correction (-37% error), spectral sharing with per-layer rotations (-99.9% error). Confirms the coupling geometry is shared; only the orientation varies.

Smooth rotation hypothesis — falsified. Tested whether inter-layer eigenvector rotations form a smooth path (parameterisable by a single angular velocity generator). Mean cosine similarity between consecutive inter-layer rotations: 0.003. The rotations are unrelated—no cheap generator exists. This motivates the implicit rotation mechanism (Section 6) rather than explicit per-layer rotation parameters.

9.4 Summary

Claim	Evidence	Status
Coupling is spectrally invariant	Exp 0: correlation > 0.97	Confirmed
Shared $\mathbf{M}$ matches per-layer QK	Exp 1: -0.7% , Exp B1: -2.6%	Confirmed
Asymmetry is essential	Exp B1: symmetric $+3.6\%$ worse	Confirmed
Scaling advantage exists	Exp 2: β ratio $1.02\times$	Modest
Orbital embeddings work end-to-end	Exp 5a/b/c: $+7.7\%$ (2M), -3.0% (10M), $+4.7\%$ ($d_s=64$)	Mixed*
Spectral sharing is lossless	Gap test 6: -99.9% error	Confirmed
Two-system architecture	—	Untested

*Mixed: $d_{\text{seed}}=64$ wins at 10M, but the mechanism may be the expansion ratio ($d/d_s \approx 5$) rather than iterative dynamics. Critical test: LowRank at $d_s=64$, $T=0$.

10 Honest Assessment

What we know. The compressed representation exists—proven by existence (every human who learned a language). Spectral coupling invariance is real (> 0.97 correlation). Asymmetric shared $\mathbf{M}$ beats the transformer on Shakespeare and TinyStories. Orbital embeddings train from scratch with no curriculum. The capacity argument (parameters $\times$ iterations $\geq$ circuit complexity) is a theorem, not speculation.

What we don’t know. All coupling experiments use synthetic or small-scale tasks; natural language at scale is untested. The scaling advantage (β ratio $1.02\times$) is modest at toy scale. The two-system architecture (orbital + ontological) is conjectural—no experiment yet. Whether deeper orbits ($T \geq 2$) help at larger scale is unknown. The output bottleneck ($d_{\text{seed}} = 16$ captures only 50% of transition variance) may understate the orbital advantage.

Kill criteria.

Experiment	Kill if	Interpretation
Coupling invariance	Spectral corr < 0.5	Coupling rules are not spectrally invariant
Symmetric coupling	All variants $> 30\%$ worse	Symmetric geometry too restrictive
Eigenspectrum	Condition number < 2	$\mathbf{M}$ collapsed to scaled identity
Scaling curves	$\beta_{\text{ORN}} \leq \beta_{\text{transformer}}$	No scaling advantage
Ontological collapse	$\ U_w\ $ large for all tokens	Model uses ontological layer for everything

The prediction is not that the ORN will beat the transformer immediately, but that it will scale differently—steeper β —because it stores generators where the transformer stores trajectories.

10.1 Future Direction: RG Diagnostics as a Measurement Framework

Independent of whether the RG analogy is deep or merely suggestive, it provides a **toolkit of measurable diagnostics** for understanding SharedM that are not available for standard transformers:

- **Fixed-point convergence:** track $\mathbf{M}$ ’s singular value spectrum over training checkpoints. Does it stabilise? When? How fast relative to the loss curve?
- **Correlation length:** compute $\xi = -1/\ln(\sigma_2/\sigma_1)$ and compare to the depth at which the residual stream empirically stabilises. If they match, the RG picture is quantitatively predictive.
- **Spectral distribution shape:** fit σ_i vs. i on a log-log plot. A power law ($\sigma_i \propto i^{-\alpha}$) would indicate the system operates near criticality—no characteristic scale, every singular direction matters at some depth. An exponential decay indicates subcriticality (ordered, fast convergence, limited capacity). A flat distribution indicates supercriticality (disordered, slow convergence).
- **Condition number dynamics:** $\kappa(t)$ over training steps. A sharp transition would confirm the phase transition prediction.

These measurements are straightforward—a few lines of logging in the training loop—and they apply specifically to SharedM because $\mathbf{M}$ is a single, inspectable object. In the standard transformer, the coupling geometry is spread across $2L$ independent matrices with no unified spectral signature to measure.

A speculative extension: if $\mathbf{M}$ ’s spectrum turns out to be power-law, it would connect the ORN to the critical brain hypothesis [36]—the observation that biological neural networks appear to operate near a critical point, maximising information transmission and dynamic range. Whether gradient descent on SharedM naturally tunes toward criticality is an empirical question answerable with existing checkpoints. The claim is not that criticality is required for the architecture to work, but that the RG framework provides the right language and measurements for understanding what $\mathbf{M}$ learns.

11 Conclusion

The transformer stores knowledge extensionally: one vector per token, one coupling matrix per layer. This is the brute-force solution—throw memory at the problem, let gradient descent sort it out. It works, but it is the wrong representation.

The ORN replaces both lookup structures with intensional representations: orbital embeddings (seeds + shared dynamics) for the embedding table, and a shared metric tensor ($\mathbf{M} = \mathbf{A}\mathbf{B}^\top$) for the coupling projections. The response heads remain per-layer, carrying the dual burden of content transformation and implicit coordinate rotation.

The theoretical framework connects three ideas:

1. **From computer science:** memoisation of pure functions that are redundantly recomputed.
2. **From biology:** stigmergic coordination through a shared field whose geometry is known.
3. **From physics:** renormalisation group flow toward compact generators through iterated coarse-graining.

The capacity argument is precise: parameters trade for iterations at constant circuit complexity. The geometric picture is clear: tokens are orbits in a phase space defined by $\mathbf{M}$, functional equivalence is orbital convergence, and the response heads co-evolve with the coupling through the shared residual field.

The two-system conjecture—orbital world model for structure, ontological layer for discrete bindings—awaits experimental test. If validated, it implies a modular architecture where adaptation (RAG, fine-tuning) targets the cheap ontological surface while the expensive orbital core remains frozen.

The experimental programme is designed to falsify each claim independently. Our preliminary results show that the ORN can match or beat the standard transformer at 10M parameters (−4.7% val loss with $d_{\text{seed}} = 64$, Experiment 5c). But we want to be clear: **beating the transformer is not the point, and we do not yet know how this result scales.** The transformer is an extraordinarily well-optimised architecture backed by years of engineering. Claiming superiority on the basis of toy-scale experiments would be premature and dishonest.

What we *are* claiming is more specific and, we believe, more interesting:

1. We can achieve **comparable results with far fewer coupling parameters**— $2d^2$ shared vs. $2Ld^2$ per-layer—by replacing stored coupling with orbital computation through a learned semigroup.
2. In doing so, we learn something new about the **structure of the problem space itself**. The factorisation into generator (shared coupling geometry), response (per-layer value projection), and field (evolving residual stream) is not just an engineering trick—it is a structural claim about how information couples in sequential data.
3. We have **found the generators, not just the local trajectories**. The standard transformer stores L independent coupling matrices— L snapshots of what the coupling looks like at each depth. The ORN learns the single dynamical rule that *generates* all of them. This is a stronger form of knowledge: a trajectory tells you where you went; a generator tells you the law that produces all possible trajectories. Our spectral invariance measurements (on GPT-2, with the caveats noted in Section 6.1) suggest that the transformer already learns an approximately invariant coupling generator and then works around it with per-layer response heads. We are making explicit what the transformer encodes implicitly.

The prediction is not immediate superiority but different scaling: steeper β , because generators compress better than trajectories. If the scaling advantage is real, it will become visible at larger scale. If it is not, the structural insight—that coupling geometry is approximately universal and can be memoised—stands on its own as a contribution to understanding how transformers represent sequential structure.

References

- [1] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *ICLR*, 2020.
- [2] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal Transformers. *ICLR*, 2019.
- [3] A. Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv:1603.08983*, 2016.

- [4] S. Bai, J. Z. Kolter, and V. Koltun. Deep Equilibrium Models. *NeurIPS*, 2019.
- [5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural Ordinary Differential Equations. *NeurIPS*, 2018.
- [6] Y. Tay, D. Bahri, D. Metzler, D.-C. Juan, Z. Zhao, and C. Zheng. Synthesizer: Rethinking Self-Attention Transformers. *ICML*, 2021.
- [7] S. Zhai, W. Talbott, N. Srivastava, C. Huang, H. Goh, R. Zhang, and J. Susskind. An Attention Free Transformer. *arXiv:2105.14103*, 2021.
- [8] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *ICML*, 2020.
- [9] H. Ramsauer et al. Hopfield Networks is All You Need. *ICLR*, 2021.
- [10] T. Cohen and M. Welling. Group Equivariant Convolutional Networks. *ICML*, 2016.
- [11] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges. *arXiv:2104.13478*, 2021.
- [12] M. Finzi, S. Stanton, P. Izmailov, and A. G. Wilson. Generalizing Convolutional Neural Networks for Equivariance to Lie Groups. *ICML*, 2020.
- [13] P. Mehta and D. J. Schwab. An exact mapping between the Variational Renormalization Group and Deep Learning. *arXiv:1410.3831*, 2014.
- [14] H. W. Lin, M. Tegmark, and D. Rolnick. Why Does Deep and Cheap Learning Work So Well? *Journal of Statistical Physics*, 168(6):1223–1247, 2017.
- [15] J. L. McClelland, B. L. McNaughton, R. C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- [16] Y. Chen, H. Wu. Deep-OSG: Deep Learning of Operators in Semigroup. *J. Computational Physics*, 2023.
- [17] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.
- [18] Fourier-Based Token Embeddings: Computed Representations without Learned Lookup Tables. *arXiv:2505.02266*, 2025.
- [19] Conditional Memory via Scalable Lookup (Engram). *arXiv:2601.07372*, 2026.
- [20] M. Hauser, A. Ray. Principles of Riemannian Geometry in Neural Networks. *NeurIPS*, 2017.
- [21] How Training Shapes the Riemannian Geometry of Representations. *arXiv:2301.11375*, 2023.
- [22] A. Benfenati, L. Marta. Singular Riemannian Geometry of Deep Neural Networks. *Neural Networks*, 2022.
- [23] I. Katsman et al. Riemannian Residual Neural Networks. *NeurIPS*, 2023.
- [24] A. Ansuini et al. Intrinsic Dimension of Data Representations in Deep Neural Networks. *NeurIPS*, 2019.

- [25] S. Amari. Information Geometry and Its Applications to Machine Learning. *arXiv:1808.07172*, 2019.
- [26] P. Shaw et al. Bridging Kolmogorov Complexity and Deep Learning: Asymptotically Optimal Description Length for Transformers. *ICLR*, 2026.
- [27] S. Lotfi et al. Unlocking Tokens as Data Points for Generalization Bounds on Larger Language Models. *NeurIPS Spotlight*, 2024.
- [28] D. de Moss et al. Complexity Dynamics of Grokking. *arXiv:2412.09810*, 2024.
- [29] A. Vacher. Deep Neural Networks as Iterated Function Systems. *arXiv:2601.19958*, 2026.
- [30] Neural ODE Transformers: Continuous-Depth Models with Learned Control Dynamics. *ICLR*, 2025.
- [31] I. McGilchrist. *The Master and His Emissary: The Divided Brain and the Making of the Western World*. Yale University Press, 2009.
- [32] I. McGilchrist. *The Matter with Things: Our Brains, Our Delusions, and the Unmaking of the World*. Perspectiva Press, 2021.
- [33] J. Kaplan, S. McCandlish, T. Henighan, et al. Scaling Laws for Neural Language Models. *arXiv:2001.08361*, 2020.
- [34] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. *NeurIPS*, 2022.
- [35] Y. Levine, N. Wies, O. Sharir, H. Bata, and A. Shashua. Limits to Depth Efficiencies of Self-Attention. *NeurIPS*, 2020.
- [36] J. M. Beggs and D. Plenz. Neuronal Avalanches in Neocortical Circuits. *Journal of Neuroscience*, 23(35), 2003.
- [37] C. Bodnar et al. Neural Sheaf Diffusion: A Topological Perspective on Heterophilic Graph Learning. *NeurIPS*, 2022.
- [38] K. Mingard, J. Rees, G. Valle-Pérez, and A. Louis. Deep Neural Networks Have an Inbuilt Occam’s Razor. *Nature Communications*, 2025.
- [39] Y. Wang et al. MASA: Share Your Attention via Matrix-based Dictionary Learning. *arXiv:2508.04581*, 2025.